convex_hull

```
// convex_hull(a,include_collinear): replaces a with its convex hull in counter-clockwise order, starting at the smallest point
// include_collinear: also keep points lying on hull edges
// cross(a,b,c): >0 if a->b->c turns left, <0 if right, 0 if collinear
struct pt{
  ll x,y;
  bool operator<(pt o)const{return pll(x,y)<pll(o.x,o.y);}
  bool operator==(pt o)const{return x==o.x&&y==o.y;}
};
ll cross(pt a,pt b,pt c){return(b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x);}

void convex_hull(vc<pt>&a,bool include_collinear=0){
  sort(all(a));a.erase(unique(all(a)),a.end());
  if(sz(a)<3)return;
  vc<pt>h;
  rep(it,2){
    int s=sz(h);
    for(pt p:a){
      while(sz(h)>=s+2&&cross(h[sz(h)-2],h.back(),p)<!include_collinear)h.pop_back();
      h.pb(p);
    }
    h.pop_back();reverse(all(a));
  }
  if(sz(h)>sz(a))h.resize(sz(a));
  a=h;
}
```

euclid

```
// gcd(a,b,x,y): returns gcd(a,b) and sets x,y so that a*x+b*y=gcd(a,b)
ll gcd(ll a,ll b,ll&x,ll&y){
  if(!b)return x=1,y=0,a;
  ll d=gcd(b,a%b,y,x);
  y-=a/b*x;
  return d;
}
```

fenwick

```
// FT(n): n zeros, upd(i,x): a[i]+=x, qry(i): a[0]+...+a[i-1]
struct FT{
  vl s;
  FT(int n):s(n){}
  void upd(int i,ll x){
    for(;i<sz(s);i|=i+1)s[i]+=x;
  }
  ll qry(int i){
    ll r=0;
    for(;i>0;i&=i-1)r+=s[i-1];
    return r;
  }
};

// FT2(h): 2D sums over points with 0<=x<h, all points that will be updated must be known in advance
// inform(x,y) for every such point, then init(), then use upd and qry
// upd(x,y,d): value at (x,y)+=d
// qry(x,y): sum over x'<x,y'<y, qry(x1,y1,x2,y2): sum over x1<=x'<x2,y1<=y'<y2
struct FT2{
  vc<vi>ys;vc<FT>ft;
  FT2(int h):ys(h){}
  void inform(int x,int y){
    for(;x<sz(ys);x|=x+1)ys[x].pb(y);
  }
  void init(){
    for(vi&v:ys)sort(all(v)),v.erase(unique(all(v)),v.end()),ft.pb(sz(v));
  }
  int ind(int x,int y){
    return lower_bound(all(ys[x]),y)-ys[x].begin();
  }
  void upd(int x,int y,ll dif){
    for(;x<sz(ys);x|=x+1)ft[x].upd(ind(x,y),dif);
  }
  ll qry(int x,int y){
    ll r=0;
    for(;x;x&=x-1)r+=ft[x-1].qry(ind(x-1,y));
    return r;
  }
  ll qry(int x1,int y1,int x2,int y2){
    return qry(x2,y2)-qry(x2,y1)-qry(x1,y2)+qry(x1,y1);
  }
};
```

fft

```
// multiply(a,b): coefficients of the product of polynomials a and b
// exact while (sum of a[i]^2 + sum of b[i]^2)*log2(n) < 9e14
typedef complex<double> C;
void fft(vc<C>&a){
  int n=sz(a),L=__builtin_ctz(n);
  static vc<complex<long double>>R(2,1);
  static vc<C>rt(2,1);
  for(static int k=2;k<n;k*=2){
    R.resize(n);rt.resize(n);
    auto x=polar(1.0L,acosl(-1)/k);
    fwd(i,k,2*k)rt[i]=R[i]=i&1?R[i/2]*x:R[i/2];
  }
  vi rev(n);
  rep(i,n)rev[i]=(rev[i/2]|(i&1)<<L)/2;
  rep(i,n)if(i<rev[i])swap(a[i],a[rev[i]]);
  for(int k=1;k<n;k*=2)for(int i=0;i<n;i+=2*k)rep(j,k){
    C z=rt[j+k]*a[i+j+k];
    a[i+j+k]=a[i+j]-z;
    a[i+j]+=z;
  }
}

vl multiply(const vi&a,const vi&b){
  if(a.empty()||b.empty())return{};
  vl res(sz(a)+sz(b)-1);
  int n=1;
  while(n<sz(res))n*=2;
  vc<C>in(n),out(n);
  rep(i,sz(a))in[i].real(a[i]);
  rep(i,sz(b))in[i].imag(b[i]);
  fft(in);
  for(C&x:in)x*=x;
  rep(i,n)out[i]=in[-i&(n-1)]-conj(in[i]);
  fft(out);
  rep(i,sz(res))res[i]=llround(imag(out[i])/(4*n));
  return res;
}
```

hash_string

```
// FastString(s): get(l,r) returns hash of s[l..r-1], equal substrings have equal hashes
typedef unsigned long long ull;
const ull HM=(1ull<<61)-1,HB=chrono::steady_clock::now().time_since_epoch().count()%HM;
ull hmul(ull a,ull b){
  __uint128_t c=(__uint128_t)a*b;
  ull r=(c&HM)+(c>>61);
  return r>=HM?r-HM:r;
}

struct FastString{
  vc<ull>h,p;
  FastString(const string&s):h(sz(s)+1),p(sz(s)+1,1){
    rep(i,sz(s))h[i+1]=(hmul(h[i],HB)+s[i])%HM,p[i+1]=hmul(p[i],HB);
  }
  ull get(int l,int r){
    return(h[r]+HM-hmul(h[l],p[r-l]))%HM;
  }
};
```

li_chao

```
// LCT(n): set of lines on integer x in [0,n)
// add_line({k,m}): adds line y=k*x+m, get(x): minimum of all lines at x (1e18 if there are none)
struct LCT{
  int n;
  vc<pll>t;
  LCT(int n):n(n),t(4*n,{0,(ll)1e18}){}
  ll ev(pll a,ll x){return a.f1*x+a.f2;}
  void add_line(pll w,int v=1,int l=0,int r=0){
    if(!r)r=n;
    int m=(l+r)/2;
    bool L=ev(w,l)<ev(t[v],l),M=ev(w,m)<ev(t[v],m);
    if(M)swap(t[v],w);
    if(r-l>1)L!=M?add_line(w,2*v,l,m):add_line(w,2*v+1,m,r);
  }
  ll get(int x,int v=1,int l=0,int r=0){
    if(!r)r=n;
    int m=(l+r)/2;
    ll y=ev(t[v],x);
    return r-l==1?y:min(y,x<m?get(x,2*v,l,m):get(x,2*v+1,m,r));
  }
};
```

max_flow

```
// MaxFlow(n): graph on nodes 0..n-1
// add(a,b,c): directed edge a->b with capacity c
// maxflow(s,t): returns max flow from s to t, call only once
// after maxflow, d[v]>=0 iff v is on the source side of a min cut
struct MaxFlow{
  struct E{int to;ll c;};
  vc<E>e;vc<vi>g;vi d,it;
  MaxFlow(int n):g(n),d(n),it(n){}
  void add(int a,int b,ll c){
    g[a].pb(sz(e));e.pb({b,c});
    g[b].pb(sz(e));e.pb({a,0});
  }
  ll dfs(int u,int t,ll f){
    if(u==t||!f)return f;
    ll r=0;
    for(int&i=it[u];i<sz(g[u]);i++){
      E&x=e[g[u][i]];
      if(d[x.to]==d[u]+1)if(ll p=dfs(x.to,t,min(f-r,x.c))){
        x.c-=p;e[g[u][i]^1].c+=p;
        if((r+=p)==f)return r;
      }
    }
    return r;
  }
  ll maxflow(int s,int t){
    ll f=0;
    while(1){
      fill(all(d),-1);fill(all(it),0);
      d[s]=0;queue<int>q;q.push(s);
      while(sz(q)){
        int u=q.front();q.pop();
        for(int i:g[u])if(e[i].c&&d[e[i].to]<0)d[e[i].to]=d[u]+1,q.push(e[i].to);
      }
      if(d[t]<0)return f;
      f+=dfs(s,t,LLONG_MAX);
    }
  }
};
```

min_cost_flow

```
// MinCostFlow(n): graph on nodes 0..n-1
// add_edge(a,b,cap,cost): directed edge a->b
// min_cost_flow(s,t,K): sends at most K flow from s to t as cheaply as possible, returns {flow,cost}
// negative costs are allowed if there is no negative cycle
struct MinCostFlow{
  struct E{int to;ll c,w;};
  int n;vc<E>e;vc<vi>g;
  MinCostFlow(int n):n(n),g(n){}
  void add_edge(int a,int b,ll cap,ll cost){
    g[a].pb(sz(e));e.pb({b,cap,cost});
    g[b].pb(sz(e));e.pb({a,0,-cost});
  }
  pll min_cost_flow(int s,int t,ll K=LLONG_MAX){
    ll f=0,c=0;
    vl pi(n,LLONG_MAX);vi inq(n);
    queue<int>q;q.push(s);pi[s]=0;
    while(sz(q)){
      int u=q.front();q.pop();inq[u]=0;
      for(int i:g[u])if(e[i].c&&pi[u]+e[i].w<pi[e[i].to]){
        pi[e[i].to]=pi[u]+e[i].w;
        if(!inq[e[i].to])inq[e[i].to]=1,q.push(e[i].to);
      }
    }
    while(f<K&&pi[t]<LLONG_MAX){
      vl d(n,LLONG_MAX);vi p(n);
      priority_queue<pll,vc<pll>,greater<>>pq;pq.push({0,s});d[s]=0;
      while(sz(pq)){
        auto[du,u]=pq.top();pq.pop();
        if(du>d[u])continue;
        for(int i:g[u]){
          E&x=e[i];
          if(!x.c)continue;
          ll nd=du+x.w+pi[u]-pi[x.to];
          if(nd<d[x.to])d[x.to]=nd,p[x.to]=i,pq.push({nd,x.to});
        }
      }
      if(d[t]==LLONG_MAX)break;
      rep(v,n)if(d[v]<LLONG_MAX)pi[v]+=d[v];
      ll r=K-f;
      for(int v=t;v!=s;v=e[p[v]^1].to)r=min(r,e[p[v]].c);
      for(int v=t;v!=s;v=e[p[v]^1].to)e[p[v]].c-=r,e[p[v]^1].c+=r;
      f+=r;c+=r*pi[t];
    }
    return{f,c};
  }
};
```

modulo_int

```
// M: integer modulo MOD (MOD must be prime for division)
// supports + - * / and assignment versions, pow, == != <, cin, cout and use as key in set, map and unordered_map
const ll MOD=1e9+7;
struct M{
  ll val;
  M(ll x=0):val((x%MOD+MOD)%MOD){}
  M&operator+=(M b){if((val+=b.val)>=MOD)val-=MOD;return*this;}
  M&operator-=(M b){if((val-=b.val)<0)val+=MOD;return*this;}
  M&operator*=(M b){val=val*b.val%MOD;return*this;}
  M&operator/=(M b){return*this*=b.pow(MOD-2);}
  friend M operator+(M a,M b){return a+=b;}
  friend M operator-(M a,M b){return a-=b;}
  friend M operator*(M a,M b){return a*=b;}
  friend M operator/(M a,M b){return a/=b;}
  friend bool operator==(M a,M b){return a.val==b.val;}
  friend bool operator!=(M a,M b){return a.val!=b.val;}
  bool operator<(M b)const{return val<b.val;}
  M pow(ll e)const{M r=1,a=*this;for(;e;e/=2,a*=a)if(e&1)r*=a;return r;}
  friend ostream&operator<<(ostream&s,M m){return s<<m.val;}
  friend istream&operator>>(istream&s,M&m){ll x;s>>x;m=x;return s;}
};
template<>struct std::hash<M>{size_t operator()(M x)const{return x.val;}};
```

setup

```
/*

setxkbmap us -option caps:escape

~/.vimrc

set number relativenumber tabstop=2 shiftwidth=2 autoindent
syntax enable
filetype plugin indent on

Makefile:

%: %.cpp
  g++ -std=c++20 -O2 -Wall -DLOC $< -o $@

 */
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
#define fwd(i,a,b) for(int i=(a); i<(b);i++)
#define rep(i,n) fwd(i,0,n)
#define all(x) (x).begin(), (x).end()
#define sz(x) ((int)(x).size())
#define f1 first
#define f2 second
#define pb push_back
#define vc vector
typedef vc<int> vi;
typedef vc<ll> vl;
typedef pair<int,int> pii;
typedef pair<ll,ll> pll;

#ifdef LOC
template<class A,class B>auto&operator<<(ostream&o,pair<A,B>p){return o<<"("<<p.f1<<", "<<p.f2<<")";}
template<class T>auto&operator<<(ostream&o,const vc<T>&v){o<<"{";for(int i=0;auto&x:v)o<<(i++?", ":"")<<x;return o<<"}";}
#define debug(x) cerr<<"[" #x "]: "<<x<<"\n";
#else
#define debug(x)
#endif

void solve(){
}

int main(){
  cin.tie(0);cout.tie(0);ios::sync_with_stdio(false);
  solve();return 0;
}
```

suffix_array

```
// suffix_array(s): starting indices of all suffixes of s in sorted order, s must not contain '\0' or negative chars
vi suffix_array(string s){
  s.pb(0);
  int n=sz(s);
  vi p(n),c(all(s)),t(n),cnt(max(256,n));
  iota(all(p),0);
  sort(all(p),[&](int a,int b){return c[a]<c[b];});
  for(int k=1;;k*=2){
    rep(i,n)t[i]=(p[i]-k+n)%n;
    fill(all(cnt),0);
    for(int x:c)cnt[x]++;
    partial_sum(all(cnt),cnt.begin());
    for(int i=n;i--;)p[--cnt[c[t[i]]]]=t[i];
    t[p[0]]=0;
    fwd(i,1,n)t[p[i]]=t[p[i-1]]+(pii(c[p[i]],c[(p[i]+k)%n])!=pii(c[p[i-1]],c[(p[i-1]+k)%n]));
    c.swap(t);
    if(c[p[n-1]]==n-1)break;
  }
  p.erase(p.begin());
  return p;
}

// lcp(s,p): with p=suffix_array(s), l[i] is the longest common prefix of suffixes p[i] and p[i+1]
vi lcp(const string&s,const vi&p){
  int n=sz(s),k=0;
  vi r(n),l(n-1);
  rep(i,n)r[p[i]]=i;
  rep(i,n)if(r[i]<n-1){
    int j=p[r[i]+1];
    while(max(i,j)+k<n&&s[i+k]==s[j+k])k++;
    l[r[i]]=k;
    k-=!!k;
  }else k=0;
  return l;
}
```

tarjan_bridge

```
// define maxN, add undirected edges to nodes[], then call tarjan_bridge(root)
// bridges then holds every bridge {u,v} of the component of root, parallel edges are handled
int timer,tin[maxN],low[maxN];
vi nodes[maxN];
vc<pii>bridges;

void tarjan_bridge(int u=0,int p=-1){
  tin[u]=low[u]=++timer;
  bool s=0;
  for(int v:nodes[u]){
    if(v==p&&!s)s=1;
    else if(tin[v])low[u]=min(low[u],tin[v]);
    else{
      tarjan_bridge(v,u);
      low[u]=min(low[u],low[v]);
      if(low[v]>tin[u])bridges.pb({u,v});
    }
  }
}
```

z_algorithm

```
// z_function(s): z[i] is the length of the longest common prefix of s and s[i..], z[0]=0
vi z_function(const string&s){
  int n=sz(s),l=0,r=0;
  vi z(n);
  fwd(i,1,n){
    if(i<r)z[i]=min(r-i,z[i-l]);
    while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
    if(i+z[i]>r)l=i,r=i+z[i];
  }
  return z;
}
```